Ad Soyad: _____	**Tarih:** _____
Lab 10 – Hafta 9 – *Çok Kanallı Programlama: Thread, Mutex ve Atomic*	

Lab Hakkında

Bu laboratuvar çalışmasında Bölüm 9'da ele alınan çok kanallı programlama kavramlarını uygulamalı olarak deneyeceksiniz. `std::thread` ile paralel iş parçacıkları oluşturacak, data race sorunlarını gözlemleyecek, `std::mutex` ve `std::lock_guard` ile bu sorunları çözecek ve son olarak `std::atomic` ile kilitsiz eşzamanlı erişim gerçekleştireceksiniz. Tüm çalışmalar saf C++20 üzerinde ilerler; Qt bağımlılığı yoktur.

Süre: 2 saat **Gereksinimler:** g++ (C++20 desteği), `-pthread` bayrağı

Derleme Komutu

Bu labtaki tüm örnekleri aşağıdaki komutla derleyebilirsiniz:

```
g++ -std=c++20 -pthread -o program program.cpp
./program
```

`-pthread` bayrağı olmadan `std::thread` bağlama hatası alırsınız. macOS'ta Clang varsayılan olarak thread desteği sunar ancak yine de `-pthread` bayrağını eklemeniz önerilir.

1 Çalışma 1: Thread Oluşturma ve join (25 dk)

Ön Bilgi

`std::thread` bir fonksiyonu veya callable nesneyi ayrı bir iş parçacığında çalıştırır. Oluşturulan her thread'in ya `join()` ile beklenmesi ya da `detach()` ile serbest bırakılması gerekir; aksi halde program `std::terminate` ile sonlanır. Bu çalışmada bir diziyi eşit parçalara bölüp her parçanın toplamını ayrı bir thread'de hesaplayacağız.

Adım 1: Temel Thread Oluşturma

Aşağıdaki kodu `calisma1.cpp` olarak kaydedin ve derleyin:

Listing 1: Kısmi Toplam Hesaplama

```cpp
#include <iostream>
#include <thread>
#include <vector>
#include <numeric>

void kismi_topla(const std::vector<int>& veri,
                 int baslangic, int bitis,
                 long long& sonuc)
{
    sonuc = 0;
    for (int i = baslangic; i < bitis; ++i) {
        sonuc += veri[i];
    }
}

int main() {
    constexpr int N = 1'000'000;
```

```
    std::vector<int> veri(N);
    std::iota(veri.begin(), veri.end(), 1); // 1, 2, ..., N

    constexpr int THREAD_SAYISI = 4;
    int parca = N / THREAD_SAYISI;

    std::vector<std::thread> izlekler;
    std::vector<long long> sonuclar(THREAD_SAYISI, 0);

    for (int i = 0; i < THREAD_SAYISI; ++i) {
        int bas = i * parca;
        int son = (i == THREAD_SAYISI - 1) ? N : bas + parca;
        izlekler.emplace_back(kismi_topla,
            std::cref(veri), bas, son,
            std::ref(sonuclar[i]));
    }

    for (auto& t : izlekler) {
        t.join();
    }

    long long toplam = 0;
    for (auto s : sonuclar) {
        toplam += s;
    }
    std::cout << "Toplam: " << toplam << '\n';
    // Beklenen: N*(N+1)/2 = 500000500000
    return 0;
}
```

Gorev 1: Kısmi Toplamları Doğrulayın

- Programı derleyip çalıştırın. Çıktının `500000500000` olduğunu doğrulayın.
- `THREAD_SAYISI` değerini 1, 2, 4 ve 8 olarak değiştirip her seferinde sonucun aynı olduğunu kontrol edin.
- Her thread'in hangi aralığı işlediğini görmek için `kismi_topla` fonksiyonuna bir `std::cout` satırı ekleyin: "Thread [baslangic, bitis) toplam = sonuc".

Gorev 2: Zaman Olcumu

- `<chrono>` başlık dosyasını kullanarak toplam çalışma süresini ölçün. `std::chrono::high_resolution_clock` ile başlangıç ve bitiş zamanlarını alın.
- 1 thread, 2 thread ve 4 thread için süreleri karşılaştırın. Sonuçları aşağıdaki tabloya yazın:

Thread Sayısı	Süre (ms)
1	
2	
4	
8	

Zamanlama İpucu

```
auto t0 = std::chrono::high_resolution_clock::now();
// ... islem ...
auto t1 = std::chrono::high_resolution_clock::now();
auto ms = std::chrono::duration_cast<
    std::chrono::milliseconds>(t1 - t0).count();
std::cout << "Sure: " << ms << " ms\n";
```

2 Çalışma 2: Data Race Gözlemleme (25 dk)

Ön Bilgi

Birden fazla thread aynı bellek adresine eşzamanlı olarak yazarsa ve bu erişimler arasında bir senkronizasyon mekanizması yoksa **data race** oluşur. C++ standardına göre data race **tanımsız davranış** (undefined behavior) demektir; program her çalıştırıldığında farklı sonuç üretebilir.

Adım 1: Yarışan Sayaç

Aşağıdaki kodu `calisma2.cpp` olarak kaydedin:

Listing 2: Korumasız Sayaç – Data Race

```
#include <iostream>
#include <thread>

int sayac = 0;

void arttir(int miktar) {
    for (int i = 0; i < miktar; ++i) {
        ++sayac; // [!] Data race!
    }
}

int main() {
    constexpr int N = 1'000'000;

    std::thread t1(arttir, N);
    std::thread t2(arttir, N);

    t1.join();
    t2.join();

    std::cout << "Beklenen: " << 2 * N << '\n';
    std::cout << "Gercek  : " << sayac << '\n';
    return 0;
}
```

Gorev 1: Data Race'i Gözlemleyin

- Programı derleyip en az **5 kez** çalıştırın.
- Her çalıştırmadaki `sayac` değerini aşağıdaki tabloya yazın:

Deneme 1	Deneme 2	Deneme 3	Deneme 4	Deneme 5	Beklenen
					2 000 000

Gorev 2: Neden Farklı Sonuclar?

Aşağıdaki soruları cevaplayın:

1. `++sayac` işlemi neden atomik değildir? (Kaç makine komutu içerir?)
2. Sonuçlar her zaman beklenen değerden küçük mü? Neden?
3. Thread sayısını 4'e çıkarırsanız (4 thread, her biri N artırma) sonuç nasıl değişir?

Derleyici Optimizasyonu

`-O2` bayrağı ile derlerseniz derleyici optimizasyonları data race'in görünürlüğünü değiştirebilir. Hem `-O0` hem `-O2` ile deneyip farkı gözlemleyin.

3 Çalışma 3: Mutex ile Düzeltme (30 dk)

Ön Bilgi

`std::mutex` bir kaynağa aynı anda yalnızca bir thread'in erişmesini sağlar. `lock()` ile kilit alınır, `unlock()` ile bırakılır. Ancak doğrudan `lock/unlock` kullanmak istisna güvensizdir; `std::lock_guard` RAII sarmalayıcısı otomatik olarak kilidi kapsamdan çıkınca serbest bırakır. Daha esnek senaryolarda `std::unique_lock` tercih edilir.

Adım 1: lock_guard ile Koruma

Çalışma 2'deki kodu `calisma3.cpp` olarak kopyalayıp aşağıdaki değişiklikleri yapın:

Listing 3: Mutex ile Korunan Sayaç

```
#include <iostream>
#include <thread>
#include <mutex>

int sayac = 0;
std::mutex mtx;

void arttir(int miktar) {
    for (int i = 0; i < miktar; ++i) {
        std::lock_guard<std::mutex> kilit(mtx);
        ++sayac;
    }
```

```
}

int main() {
    constexpr int N = 1'000'000;

    std::thread t1(arttir, N);
    std::thread t2(arttir, N);

    t1.join();
    t2.join();

    std::cout << "Beklenen: " << 2 * N << '\n';
    std::cout << "Gercek  : " << sayac << '\n';
    return 0;
}
```

Gorev 1: Doğru Sonucu Doğrulayın

- Programı 5 kez çalıştırın. Her seferinde sonucun 2 000 000 olduğunu doğrulayın.
- Çalışma 2'deki korumasız versiyon ile süreleri karşılaştırın. Mutex versiyonu kaç kat yavaş?

Versiyon	Süre (ms)
Korumasız (Çalışma 2)	
`lock_guard`	

Gorev 2: unique_lock ve Ertelenmiş Kilitleme

- `lock_guard` yerine `std::unique_lock` kullanarak aynı korumayı sağlayın.
- Ardından `std::defer_lock` parametresini ekleyerek kilidi ertelenmiş şekilde oluşturun ve `kilit.lock()` ile manuel kilitleyin:

```
std::unique_lock<std::mutex> kilit(mtx, std::defer_lock);
// ... bazi hesaplamalar (kilit yok) ...
kilit.lock();
++sayac;
kilit.unlock();
```

Bu yaklaşımın `lock_guard`'a göre avantajı ne olabilir? Cevabınızı yazın:

Gorev 3: Toplu Artırma Optimizasyonu

Her iterasyonda kilit almak yerine, her thread'in yerel bir sayaçta topladıktan sonra sadece bir kez mutex alıp genel sayaca eklemesini sağlayın. Süreyi ölçün ve döngü içi kilitleme ile karşılaştırın.

Kilit Kapsamı

Kilit kapsamını mümkün olduğunca dar tutun. Döngünün tamamını kilitleyen bir `lock_guard` thread'leri seri hale getirir ve paralellikten fayda sağlanamaz. Sadece paylaşılan veriye erişen satırları kilitleyin.

4 Çalışma 4: OBYS Arka Plan Kaydetme (30 dk)

Ön Bilgi

Gerçek uygulamalarda dosyaya yazma gibi yavaş işlemler ana iş parçacığını bloke etmemelidir. Kitapta gördüğümüz `BackgroundSaver` modeli, verilerin bir anlık kopyasını (snapshot) alıp arka planda diske yazan bir thread kullanır. Bu çalışmada basitleştirilmiş bir versiyonunu `std::atomic<bool>` ve `std::mutex` ile gerçekleştireceğiz.

Adım 1: Veri Modeli

Aşağıdaki basit OBYS veri yapısını kullanacağız:

Listing 4: Basit OBYS Veri Modeli

```
#include <iostream>
#include <thread>
#include <mutex>
#include <atomic>
#include <vector>
#include <string>
#include <fstream>
#include <chrono>

struct Ogrenci {
    int numara;
    std::string ad;
    double ortalama;
};

class ArkaplanKaydedici {
public:
    void veriGuncelle(const std::vector<Ogrenci>& yeniVeri) {
        std::lock_guard<std::mutex> kilit(mtx_);
        veri_ = yeniVeri; // Snapshot al
    }

    void kaydet(const std::string& dosyaAdi) {
        kaydediliyor_.store(true);

        std::vector<Ogrenci> kopya;
        {
            std::lock_guard<std::mutex> kilit(mtx_);
            kopya = veri_; // Kilit altında kopyala
        }
        // Kilit serbest, dosyaya yazma kilitsiz
        std::ofstream dosya(dosyaAdi);
        for (const auto& o : kopya) {
            dosya << o.numara << ","
                  << o.ad << ","
                  << o.ortalama << "\n";
        }
```

```
        // Yavas disk simulasyonu
        std::this_thread::sleep_for(
            std::chrono::milliseconds(500));

        kaydediliyor_.store(false);
    }

    bool kaydediliyorMu() const {
        return kaydediliyor_.load();
    }

private:
    std::vector<Ogrenci> veri_;
    std::mutex mtx_;
    std::atomic<bool> kaydediliyor_{false};
};
```

Gorev 1: ArkaplanKaydedici'yi Tamamlayın

- Yukarıdaki kodu `calisma4.cpp` dosyasına kopyalayın.
- Aşağıdaki `main` fonksiyonunu ekleyin:

```
int main() {
    ArkaplanKaydedici kaydedici;

    std::vector<Ogrenci> ogrenciler = {
        {101, "Ali", 3.2},
        {102, "Ayse", 3.8},
        {103, "Mehmet", 2.9}
    };
    kaydedici.veriGuncelle(ogrenciler);

    // Arka planda kaydet
    std::thread kayitThread(
        &ArkaplanKaydedici::kaydet,
        &kaydedici, "ogrenciler.csv");

    // Ana thread calismaya devam eder
    std::cout << "Kayit basladi...\n";
    while (kaydedici.kaydediliyorMu()) {
        std::cout << "." << std::flush;
        std::this_thread::sleep_for(
            std::chrono::milliseconds(100));
    }
    std::cout << "\nKayit tamamlandi!\n";

    kayitThread.join();
    return 0;
}
```

Programı derleyip çalıştırın. `ogrenciler.csv` dosyasının oluştuğunu doğrulayın.

Gorev 2: Eşzamanlı Güncelleme

- Kayıt thread'i çalışırken ana thread'den `veriGuncelle()` çağırarak listeye yeni bir öğrenci ekleyin.
- Dosyaya yazılan verinin güncellemeden **önceki** anlık kopyayı mı yoksa sonraki halini mi içerdiğini gözlemleyin.
- Bu davranışın neden doğru olduğunu açıklayın (snapshot tutarlılığı):

Gorev 3: Durum Bildirimi

- `ArkaplanKaydedici` sınıfına bir `std::atomic<int> kayitSayisi_` alanı ekleyin.
- Her başarılı kaydetme işleminden sonra bu sayacı artırın.
- `main` fonksiyonunda kaydetme tamamlandıktan sonra toplam kayıt sayısını yazdırın.

atomic ve mutex Farkı

`std::atomic<bool>` tek bir değişken için kilitsiz (lock-free) senkronizasyon sağlar. Birden fazla alanı tutarlı şekilde güncellemek gerektiğinde ise `mutex` kullanılmalıdır. Bu örnekte veri vektörü mutex ile, durum bayrağı atomic ile korunmaktadır.

Bonus: Atomic Sayaç Karşılaştırması (Opsiyonel)

Bonus

Aşağıdaki görev zorunlu değildir; bitirenler ekstra puan alır.

Bonus: Mutex vs. Atomic Kıyaslama

Çalışma 3'teki sayaç problemini üç farklı yöntemle çözün ve sürelerini karşılaştırın:

1. `std::mutex` + `lock_guard` (Çalışma 3'teki gibi)
2. `std::atomic<int>` ile `++sayac`
3. `std::atomic<int>` ile `sayac.fetch_add(1, std::memory_order_relaxed)`

Yöntem	Süre (ms)	Doğru Sonuç?
`mutex + lock_guard`		☐
`atomic (varsayılan)`		☐
`atomic (relaxed)`		☐

Hangi yöntem en hızlı? `memory_order_relaxed` kullanmanın burada bir sakıncası var mı? Neden?

Teslim Bilgileri

Teslim

- Tüm `.cpp` dosyalarını ve ekran görüntülerini sıkıştırıp sisteme yükleyin.
- Dosya adı formatı: `Lab10_OgrenciNo_AdSoyad.zip`
- Her çalışma için derleme çıktısını ve program sonucunu gösteren en az bir ekran görüntüsü ekleyin.
- Teslim tarihi: Lab saatinden itibaren 1 hafta.

Çalışma	Puan
Çalışma 1: Thread Oluşturma ve join	20
Çalışma 2: Data Race Gözlemleme	20
Çalışma 3: Mutex ile Düzeltme	30
Çalışma 4: OBYS Arka Plan Kaydetme	30
Bonus: Atomic Kıyaslama	+10
Toplam	**100 (+10)**

Başarılar!